

Generative Artificial Intelligence

Complete Tutorial: GenAI, LLMs, Transformers, Word2Vec, RNNs, LSTM

GARP RAI Exam Preparation

Slide 1: What is Generative AI?

Definition:

- Generative Artificial Intelligence (GenAI) is a branch of AI that employs models to generate text, sound, or videos
- Creates new content rather than just analyzing existing data
- Represents one of the most significant technological developments of the 2020s

Key Milestone:

- OpenAI released ChatGPT in November 2022
- Reached 1 million users in just 5 days (fastest AI tool ever)
- Facebook took almost a year to reach this milestone

Impact:

- Facilitated rapid adoption of Generative AI
- Transformed how businesses and individuals interact with AI
- Created new opportunities across all sectors

Key Insight: GenAI creates new content - not just analyzes!

Traditional AI/Machine Learning:

- Analyzes existing data
- Makes predictions or classifications
- Examples: Regression, classification, clustering
- Outputs are labels or numerical values
- Deterministic (same input = same output)

Generative AI:

- Creates new content
- Generates text, images, audio, video
- Examples: ChatGPT, DALL-E, Gemini
- Outputs are novel, creative content
- Probabilistic (same input may yield different outputs)

Key Difference:

- Traditional AI: "What is this?"
- Generative AI: "Create something new based on this"

Remember: GenAI creates; Traditional AI analyzes!

What is ChatGPT?

- Chatbot based on GPT (Generative Pre-trained Transformer)
- Developed by OpenAI
- Uses transformer-based LLM technology
- Mimics human conversation via text

Key Features:

- 1 Conversational interface
- 2 Can answer follow-up questions
- 3 Admits mistakes
- 4 Challenges incorrect premises
- 5 Rejects inappropriate requests

Significance:

- Made LLMs accessible to general public
- Demonstrated practical applications of GenAI
- Sparked widespread interest and adoption

Key Insight: ChatGPT popularized LLMs for everyday use!

Five Major Content Modalities:

1 Text

- Underlying technology: Transformers
- Examples: ChatGPT, Claude, Gemini

2 Image

- Underlying technology: DALL-E, Stable Diffusion
- Creates images from text prompts

3 Audio

- Includes speech, sound effects, music
- Examples: Text-to-speech, music generation

4 Video

- Creates videos from prompts
- Examples: Sora, RunwayML

5 Multimodal

- Takes multiple input types
- Generates multiple output types
- Examples: Gemini, GPT-4

Key Concept: GenAI works across multiple content types!

Key Modeling Algorithms:

① Large Language Models (LLMs)

- Billions of parameters
- Trained on vast text corpora
- Transformer-based architecture

② Transformers

- Self-attention mechanism
- Examples: BERT, GPT

③ Stable Diffusion

- Image generation
- Probabilistic reconstruction

④ Variational Autoencoders (VAE)

- Deep neural networks
- Image modification and generation

⑤ Generative Adversarial Networks (GANs)

- Generator + Discriminator
- Creates realistic synthetic data

⑥ Recurrent Neural Networks (RNNs)

- Sequential data processing
- Time series and text

Key Insight: Different algorithms for different content types!

Slide 6: DALL-E and Image Generation

DALL-E:

- Popular image generator developed by OpenAI
- Takes text prompts and creates images
- Can generate novel, creative images

Example Prompt:

- "Create an image that personifies the 'ups and downs' of financial markets as if it were investors sitting on a roller coaster"

Capabilities:

- 1 Generate images from text descriptions
- 2 Modify existing images
- 3 Combine concepts in creative ways
- 4 Understand complex prompts

Applications:

- Marketing materials
- Rapid prototyping
- Creative projects
- Presentations and illustrations

Key Insight: DALL-E creates images from text descriptions!

What is Stable Diffusion?

- Developed by Stability AI (released 2022)
- Powerful image generating technology
- Trained on database of images
- Uses probabilistic functions

How It Works:

- 1 Start with random noise
- 2 Gradually refine to match text prompt
- 3 Uses diffusion process
- 4 Reconstructs images step by step

Advantages:

- Open source
- Runs on consumer GPUs
- High quality outputs
- Customizable

Key Concept: Stable Diffusion reconstructs images from noise based on text!

Slide 8: Variational Autoencoders (VAE)

What are VAEs?

- Type of deep neural network
- Multiple hidden layers
- Used for image modality
- Particularly useful for modifying existing images

Architecture:

1 Encoding Layer

- Compresses images
- Reduces dimensionality

2 Decoding Layer

- Reconstructs images
- Uses probabilistic inference
- Fills in missing information

Applications:

- Image modification
- Anomaly detection
- Data generation
- Feature extraction

Key Insight: VAEs are excellent for modifying existing images!

Slide 9: Generative Adversarial Networks (GANs)

What are GANs?

- Used to create images, audio, and video
- Employs two competing neural networks
- Generator creates simulated data
- Discriminator distinguishes real from fake

Architecture:

1 Generator

- Creates simulated data
- Learns to fool discriminator

2 Discriminator

- Identifies real vs fake data
- Assigns probability scores
- Sends feedback to generator

Process:

- Iterative competition
- Generator improves over time
- Eventually generates indistinguishable data

Key Insight: GANs = two networks competing to create realistic content!

Relationship Overview:

- LLMs are a subset of GenAI
- GenAI is broader - includes image, audio, video
- LLMs specifically focus on text generation

GenAI Includes:

- 1 **Text Models (LLMs)**
 - ChatGPT, Claude, Gemini
- 2 **Image Models**
 - DALL-E, Stable Diffusion
- 3 **Audio Models**
 - Text-to-speech, music generation
- 4 **Video Models**
 - Sora, RunwayML
- 5 **Multimodal Models**
 - Gemini, GPT-4

Key Concept: All LLMs are GenAI, but not all GenAI are LLMs!

Three Major Limitations:

1 No Semantic Understanding

- Cannot identify similar meanings
- "Amazing" and "fantastic" treated differently

2 Sparse Vectors

- Vast vectors with mostly zeros
- Inefficient storage and analysis
- Computational burden

3 No Context

- Words treated independently
- "Call option" vs "phone call" confused
- Negation words not handled

Why This Matters:

- Limits analytical power
- Misses important linguistic relationships
- Creates computational inefficiency

Key Insight: BoW has fundamental limitations Word2Vec addresses!

What is Word2Vec?

- NLP model created by Google team (2013)
- Led by Tomas Mikolov
- Alternative to conventional BoW
- Represents text as numerical vectors

Key Concept:

- Each word has its own vector
- Constructed by examining surrounding words
- Measures "co-occurrence" of words

Advantages Over BoW:

- 1 Compressed representation
- 2 Dense vector representations
- 3 Reduces dimensionality
- 4 Captures semantic relationships

Key Insight: Word2Vec creates dense, meaningful word representations!

What are Word Embeddings?

- Dense vector representations of words
- Generated through neural network training
- Encode semantic information
- Position words in continuous vector space

Characteristics:

- 1 **Dense:** Fewer dimensions, mostly non-zero values
- 2 **Semantic:** Similar words have similar vectors
- 3 **Learnable:** Trained from data
- 4 **Contextual:** Capture word relationships

Typical Dimensions:

- Embedding layer often has 300 neurons
- Much smaller than vocabulary size
- Efficient representation

Key Concept: Embeddings = dense vectors capturing meaning!

Slide 14: Continuous Bag of Words (CBoW)

CBoW Architecture:

- "Fill in the blank" technique
- Predicts missing word from context
- Uses context words before and after

How It Works:

- 1 Set context window (size c)
- 2 Identify words before and after target ($2c$ words)
- 3 Input: context words
- 4 Output: target word
- 5 Goal: $P(w_i | w_{i-c}, \dots, w_{i+c})$

Example:

- Sentence: "bonds are less risky compared to stocks"
- Target word: "risky"
- Input: words preceding and following "risky"

Key Insight: CBoW predicts target from surrounding context!

Skip-gram Architecture:

- "Word association" technique
- Uses one word to predict surrounding words
- Maximizes joint probability of context words

How It Works:

- 1 Input: target word
- 2 Output: context words ($2c$ words)
- 3 Goal: $P(w_{i-c}, \dots, w_{i+c} | w_i)$
- 4 Tries to predict surrounding words

Example:

- Sentence: "bonds are less risky compared to stocks"
- Input: "risky"
- Output: words preceding and following "risky"

Comparison with CBoW:

- CBoW: context $\rightarrow$ target
- Skip-gram: target $\rightarrow$ context

Key Insight: Skip-gram predicts context from target word!

Network Structure:

- Similar to autoencoder
- Single hidden layer (embedding layer)
- Input: one-hot encoded vectors
- Output: probability predictions

Components:

1 Input Layer

- One-hot encoded vectors
- Size: vocabulary size ($|W|$)

2 Hidden Layer

- Embedding layer
- Typically 300 neurons
- Learns word representations

3 Output Layer

- Probability distribution over words
- Size: vocabulary size ($|W|$)

Key Insight: Hidden layer creates the word embeddings!

Slide 17: Dimensionality Reduction with Word2Vec

Comparison of Dimensions:

Method	Dimensions
Traditional BoW	$ W \times W $
Word2Vec	$ W \times 300$

Example Calculation:

- Vocabulary size: 100,000 words
- Traditional BoW: $100,000 \times 100,000 = 10$ billion weights
- Word2Vec: $100,000 \times 300 = 30$ million weights
- **Reduction:** 99.7% fewer weights!

Benefits of Reduction:

- 1 Less memory required
- 2 Faster computation
- 3 Better generalization
- 4 More efficient training

Key Insight: Word2Vec dramatically reduces dimensionality!

Training Process:

- ➊ **Feed neural network with text**
 - Large corpus of documents
 - Process word by word
- ➋ **Learn embedding weights**
 - Through backpropagation
 - Using gradient descent
- ➌ **Generate dense vectors**
 - Each word gets vector representation
 - Semantic information encoded

What the Model Learns:

- Co-occurrence patterns
- Word relationships
- Semantic meanings
- Contextual information

Key Concept: Training learns embeddings from text data!

Definition:

- Measures "distance" between words in embedding space
- Similar words close together
- Dissimilar words far apart
- Range: $[-1, 1]$

Formula:

$$\cos(\theta) = \frac{P \cdot Q}{||P|| \times ||Q||}$$

Interpretation:

Cosine Similarity	Meaning
1.0	Identical (or very similar)
0.7 - 0.9	Very similar
0.4 - 0.6	Somewhat similar
0.0 - 0.3	Dissimilar

Example:

- China and Japan: 0.84
- Gold and Silver: 0.95
- Banana and Rose: 0.30

Key Insight: Cosine similarity measures word closeness!

Slide 20: Word Similarity Examples

Similar Word Pairs:

Word 1	Word 2	Similarity
China	Japan	0.84
King	Queen	0.78
Gold	Silver	0.95
Europe	Asia	0.83

Dissimilar Word Pairs:

Word 1	Word 2	Similarity
Australia	Tokyo	0.40
King	Man	0.53
Gold	Dollar	0.50
Banana	Rose	0.30

Key Observation:

- Similar categories = high similarity
- Different categories = low similarity
- Embeddings capture meaningful relationships

Key Insight: Embeddings distinguish similar and dissimilar words!

Word Analogies:

- "a is to b" as "c is to d"
- Captures relationships between word pairs
- Operations in embedding space

Examples:	Operation	Closest Word	Similarity
	America - Washington + London	UK	0.77
	King - Man + Woman	Queen	0.86
	Gold - Dollar + Pound	Silver	0.80
	Europe - Spain + China	Asia	0.79

How It Works:

- 1 Embedding vectors capture relationships
- 2 Vector arithmetic preserves relationships
- 3 "king - man + woman equals queen"

Key Insight: Vector arithmetic reveals semantic relationships!

t-SNE Visualization:

- t-SNE (t-distributed Stochastic Neighbor Embedding)
- Reduces embeddings to 2 dimensions
- Visualizes relationships between words
- Clusters show semantic groups

What We See:

- 1 **Europe and Asia** - close together (continents)
- 2 **Spain and China** - close together (countries)
- 3 **Europe-Spain relationship** $\approx$ Asia-China relationship

Key Insights from Visualization:

- Related words cluster together
- Relationships are preserved in 2D
- Reveals structure in embeddings

Key Concept: t-SNE visualizes embedding relationships!

Major Limitations:

1 No Sentence-Level Context

- Same word has different meanings
- Example: "bat" (animal) vs "bat" (sports)
- Word2Vec gives same vector for both

2 Fixed Context Window

- Must specify window size a priori
- May miss distant relationships
- Limited by window size

3 Ignore Word Order

- Within the context window
- Loss of syntactic information

Example:

- "The bat flew out of the cave" vs "He picked up the bat to play baseball"
- Same embedding for both "bat" instances
- Context-dependent meaning is lost

Alternative Word Embeddings:

1 GloVe (Global Vectors)

- Stanford University
- Combines matrix factorization with word2vec
- Uses global co-occurrence statistics

2 Sentence Embeddings

- SBert (Sentence BERT)
- Captures sentence-level meaning
- Better for sentence comparison

3 Document Embeddings

- Doc2Vec
- Creates document-level vectors
- Captures document meaning

Gemini Embeddings:

- Advanced embedding model
- Part of Google's Gemini family
- Multimodal capabilities

Key Insight: Many embedding approaches exist for different needs!

Standard Input for NLP Systems:

1 Clustering

- Group similar documents
- Topic identification
- Document categorization

2 Sentiment Analysis

- Customer feedback analysis
- Market sentiment detection
- Social media monitoring

3 Recurrent Models

- LSTM and RNN inputs
- Sequence processing
- Text generation

4 Transformer Models

- Pre-trained embeddings
- Transfer learning
- Contextual embeddings

Key Insight: Embeddings are foundation for modern NLP!

What are RNNs?

- Recurrent Neural Networks
- Developed for sequential data
- Handle text and speech
- Laid groundwork for GenAI

Key Features:

1 Memory Cell

- Hidden cell stores information
- Value at time t depends on previous values

2 Sequential Processing

- Processes one word at a time
- Combines current inputs with memory

3 Autoregressive Nature

- Outputs become inputs for next step
- Captures dependencies

Key Insight: RNNs process sequential data with memory!

Basic RNN Structure:

- Input at time t : x_t
- Hidden state at time t : h_t
- Output at time t : y_t
- $h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$
- $y_t = g(W_{hy}h_t + b_y)$

Diagram:

- Input $\rightarrow$ Hidden Cell $\rightarrow$ Output
- Hidden cell feeds back to itself
- Previous hidden state influences current output

Advantages:

- 1 Handles variable length sequences
- 2 Captures order of words
- 3 Learns dependencies
- 4 Uses autoregressive structure

Key Concept: RNNs combine current input with previous state!

Feedforward Neural Network (FNN):

- No memory
- Input features flow one direction
- Output depends only on current inputs
- Used for static data

Recurrent Neural Network (RNN):

- Has memory (hidden state)
- Information can flow in loops
- Output depends on current inputs AND previous states
- Used for sequential data

Key Difference:

- FNN: $y = f(x)$
- RNN: $y_t = f(x_t, h_{t-1})$

Remember: RNNs have memory; FNNs do not!

Slide 29: Vanishing Gradient Problem in RNNs

The Problem:

- Gradients become very small during backpropagation
- Early layers receive tiny gradients
- Weights in early layers barely update
- Long-range dependencies are lost

Why It Happens:

- 1 Gradients multiplied through many time steps
- 2 Each multiplication by < 1
- 3 Product approaches zero
- 4 Repeated with each time step

Example:

- Sentence: "the analyst stopped using MGARCH because it is too hard to estimate"
- "MGARCH" is 7 words away from "estimate"
- RNN loses connection between them

Key Insight: Vanishing gradients limit RNN effectiveness!

Slide 30: Long Short-Term Memory (LSTM)

What is LSTM?

- Special class of RNNs
- Developed to address vanishing gradient problem
- Uses layers/gates to regulate information flow
- Can capture long-range dependencies

Key Feature:

- Learns what to remember
- Learns what to forget
- More sophisticated memory management

Advantages:

- 1 Handles long-term dependencies
- 2 More effective than basic RNNs
- 3 Successful in many applications
- 4 Used for time series, text, audio

Key Insight: LSTMs solve the vanishing gradient problem!

Three Gates in LSTM:

1 Forget Gate

- Determines what to discard from memory
- Forgets irrelevant information
- $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$

2 Input Gate

- Determines what new information to store
- Updates memory with important information
- $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$

3 Output Gate

- Determines what to output
- Uses relevant memory for output
- $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$

Key Concept: Gates control information flow in LSTM!

LSTM Cell Components:

- Input: x_t (current input), h_{t-1} (previous hidden state), c_{t-1} (previous cell state)
- Output: h_t (current hidden state), c_t (current cell state)

Gate Functions:

1 Cell State Update:

- $\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$
- Candidate values for new cell state

2 Cell State:

- $c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t$
- Combines forgetting and new information

3 Hidden State:

- $h_t = o_t \cdot \tanh(c_t)$
- Output based on filtered cell state

Key Insight: LSTM manages long-term memory effectively!

Basic RNN:

- Simple structure
- Prone to vanishing gradients
- Struggles with long-range dependencies
- One activation function

LSTM:

- Complex structure with gates
- Solves vanishing gradient
- Handles long-range dependencies
- Multiple activation functions

When to Use Each:

RNN	LSTM
Short sequences	Long sequences
Simple patterns	Complex patterns
Limited data	Large data
Quick prototyping	Production models

Key Insight: LSTM is more powerful but more complex!

Applications in Finance:

1 Time Series Analysis

- Stock price prediction
- Volatility forecasting
- Economic indicator forecasting

2 Text Processing

- Document classification
- Sentiment analysis
- Language modeling

3 Speech Recognition

- Audio-to-text conversion
- Voice commands
- Speech synthesis

Why They Work:

- Capture temporal dependencies
- Handle variable length sequences
- Learn from sequential patterns

Key Insight: RNNs/LSTMs excel at sequence-based tasks!

Key Limitations:

- ❶ **Vanishing/Exploding Gradients**
 - Limits long-range learning
 - Requires careful design
- ❷ **Sequential Processing**
 - Can't parallelize
 - Slower than transformers
- ❸ **Memory Limitations**
 - Forgets distant information
 - Limited context window

The Solution:

- Transformers address these limitations
- Self-attention mechanisms
- Parallel processing
- Better long-range dependencies

Key Insight: Transformers overcame RNN limitations!

What are Transformers?

- Next major development in NLP
- Class of feedforward neural networks
- Based on "attention" mechanism
- Introduced in "Attention is All You Need" (2017)

Key Innovation:

- Self-attention mechanism
- Multi-head attention
- Semantic understanding
- Contextual relationships

Why Transformers Matter:

- 1 Superior to RNNs for NLP tasks
- 2 Parallel processing
- 3 Captures long-range dependencies
- 4 Foundation for LLMs

Key Insight: Transformers revolutionized NLP!

Two Main Components:

1 Encoder

- Processes input text
- Converts to vectors
- "Understands" the text

2 Decoder

- Generates output text
- Based on input/prompt
- Creates responses

Evolution:

- Some transformers specialize in encoding (BERT)
- Some focus on decoding (GPT)
- Some have both (BART)

Key Components:

- Embedding layers
- Positional encoding
- Multi-head attention
- Feed-forward networks
- Layer normalization

Key Concept: Encoder processes; Decoder generates!

What is Self-Attention?

- Calculates relationships between all words
- Creates semantic understanding
- Captures long-range dependencies

How It Works:

1 Query Vector

- Current token being compared
- Asks "what should I pay attention to?"

2 Key Vector

- Other tokens being compared
- Response to the query

3 Value Vector

- Content of the token
- Used to create context vector

Key Insight: Attention determines what's important!

Step 1: Score Calculation

- Compare query and key vectors
- $score(q_i, k_j) = q_i \cdot k_j$
- Higher score = more relevant

Step 2: Softmax Normalization

- $\alpha_{ij} = \text{softmax}(score(q_i, k_j))$
- Normalizes scores to probabilities
- Ensures gradients don't vanish

Step 3: Context Vector

- $c_i = \sum_{j=1}^n \alpha_{ij} v_j$
- Weighted sum of value vectors
- Context-enriched embedding

Example:

- "debt is senior to equity, so it is less risky"
- "it" has highest attention to "debt"
- Model learns "it" refers to "debt"

Key Insight: Attention = query-key similarity + weighted values!

What is Multi-Head Attention?

- Multiple attention heads in parallel
- Each head focuses on different aspects
- Captures different relationships

How It Works:

- 1 Partition input into sub-matrices
- 2 Each head processes independently
- 3 Combine results from all heads

Example:

- GPT basic version has 12 heads
- Each head captures different patterns
- Syntax, semantics, context, etc.

Benefits:

- Richer representations
- Multiple relationship types
- Better understanding

Key Insight: Multi-head captures diverse relationships!

Slide 41: Positional Embeddings

Why Positional Information Matters:

- Word order is crucial for meaning
- "dog bites man" $\neq$ "man bites dog"
- Need to capture position in sequence

Two Approaches:

1 Absolute Positional Embeddings

- Sine and cosine functions
- $PE_{p,2i} = \sin(p/10000^{2i/d})$
- $PE_{p,2i+1} = \cos(p/10000^{2i/d})$

2 Relative Positional Embeddings

- Adds position number to attention score
- Focuses on relative distances

Parametric Approach:

- Initialize with random numbers
- Learn during training
- Flexible and adaptive

Key Insight: Positional embeddings preserve word order!

Slide 42: Transformer Advantages

Advantages over RNNs:

1 Parallel Processing

- Processes entire sequence at once
- Much faster training
- Better hardware utilization

2 Long-Range Dependencies

- Direct connections between words
- No information loss over distance
- Better semantic understanding

3 Contextualized Embeddings

- Different vectors for different contexts
- Same word, different meaning
- Handles polysemy effectively

4 Universal World Representations

- General knowledge
- Multiple tasks
- Transfer learning

Key Insight: Transformers are superior for most NLP tasks!

Key Disadvantages:

1 Computational Intensity

- More computationally intensive than RNNs
- Requires powerful hardware
- Higher energy consumption
- Longer training times

2 Interpretability Challenges

- Complex, multi-layered structure
- Harder to understand decisions
- Difficult to identify error sources
- Opaque reasoning

3 Data Requirements

- Need huge training datasets
- Billions of parameters
- Extensive computation

Key Insight: Transformers are powerful but resource-intensive!

Self-Attention:

- Words analyze other words in same sequence
- Captures internal relationships
- Example: "it" → "debt"

Cross-Attention:

- Words analyze words in another sequence
- Used in encoder-decoder models
- Example: Translation tasks

Masked Attention:

- Blocks future tokens in decoder
- Prevents cheating
- Used for next-token prediction

Historical Attention Methods:

- 1 Additive Attention (Bahdanau)
- 2 Dot-Product Attention (Luong)
- 3 Scaled Dot-Product Attention (Vaswani)

Key Concept: Different attention types for different tasks!

Encoder Layer:

1 Multi-Head Self-Attention

- Captures relationships between input tokens
- Produces context vectors

2 Add & Normalize

- Residual connection
- Prevents vanishing gradients

3 Feed-Forward Network

- Processes context vectors
- Nonlinear transformations

4 Add & Normalize

- Another residual connection
- Layer normalization

Decoder Layer:

- Similar structure plus masked attention
- Prevents seeing future tokens
- Autoregressive generation

Key Insight: Multiple layers with residual connections!

Slide 46: Transformer vs RNN Comparison

Processing Style:

- **RNN:** Sequential (word by word)
- **Transformer:** Parallel (all words at once)

Memory/Context:

- **RNN:** Limited by hidden state
- **Transformer:** Direct connections to all words

Training Speed:

- **RNN:** Slow (sequential)
- **Transformer:** Fast (parallel)

Long-Range Dependencies:

- **RNN:** Struggles (vanishing gradients)
- **Transformer:** Excels (attention)

Interpretability:

- **RNN:** More interpretable
- **Transformer:** Less interpretable

Key Insight: Transformers win on performance; RNNs on interpretability!

What is BERT?

- Bidirectional Encoder Representations from Transformers
- Introduced by Google in 2018
- Encoder-only architecture
- 100 million parameters

Key Features:

1 Bidirectional

- Examines words before and after
- Better context understanding
- More accurate representations

2 Training Methods

- Masked Language Modeling (15% masked)
- Next Sentence Prediction

3 Applications

- Classification
- Sentiment analysis
- Question answering

Key Insight: BERT is encoder-only and bidirectional!

Stage 1: Pre-training

- Trained on large unlabeled corpus
- Books and Wikipedia pages
- Learned general language understanding

Stage 2: Self-Supervision

1 Masked Language Modeling

- 15% of words randomly masked
- Model predicts hidden words
- Learns bidirectional context

2 Next Sentence Prediction

- Sentence pairs from documents
- Predict if sentences are sequential
- Understands relationships

Result:

- Deep language understanding
- Transferable knowledge
- Strong performance on NLP tasks

Key Insight: BERT learns from masked words and sentence pairs!

RoBERTa:

- Robustly Optimized BERT Pre-training Approach
- Different masking patterns each cycle
- Larger training dataset
- More training cycles
- Multi-language web crawled data

FinBERT:

- Financial text specialization
- Trained on Reuters financial documents
- Corporate statements
- Analyst reports
- Optimal for financial sentiment analysis

Key Difference:

- RoBERTa: General improvement
- FinBERT: Domain specialization

Key Insight: Specialized BERT variants for specific tasks!

What is BART?

- Bidirectional and Auto-Regressive Transformer
- Developed by Facebook (Meta) in 2019
- Encoder-Decoder architecture
- More versatile than BERT

Key Features:

1 Bidirectional Encoder

- Similar to BERT
- Understands input

2 Auto-Regressive Decoder

- Similar to GPT
- Generates text

3 Training Method

- Distorts blocks of text
- Shuffles, removes, replaces words
- Learns to generate original forms

Key Insight: BART combines BERT and GPT capabilities!

What are Large Language Models?

- Predominantly based on transformers
- Pre-trained on vast textual datasets
- Large neural networks with billions of parameters
- Trained on very large corpora

Key Characteristics:

1 Size

- Billions to trillions of parameters
- Huge computational requirements

2 Training

- Trained only once
- On massive text corpus

3 Storage

- Cloud-based (too large for local)
- Remote access

Key Insight: LLMs are huge transformer-based models!

Autoencoding LLMs:

- Encode sentences by masking input
- Predict masked words
- Example: BERT
- Good for understanding

Autoregressive LLMs:

- Predicts next word/token
- Generates text
- Example: GPT family
- Good for generation

Combination LLMs:

- Both encoder and decoder
- Most versatile
- Example: BART
- Good for many tasks

Key Insight: Different LLM types for different purposes!

Step 1: Data Preparation

- Clean training data (corpus)
- Filter and remove duplicates
- Remove noisy data and punctuation
- Resolve or remove ambiguous data
- Balance class distributions

Step 2: Tokenization

- Separate data into small parts (tokens)
- Enable efficient processing
- Store positional information

Step 3: Pre-training

- Train on unlabeled text
- Understand word relationships
- Learn language patterns

Key Insight: Training requires massive data and compute!

Step 4: Fine-tuning

- Transfer knowledge to specific tasks
- Use labeled data
- Low-shot task transfer

Step 5: Alignment

- Conform to human preferences
- Reinforce with human feedback (RLHF)
- Based on user feedback

Step 6: Decoding

- Generate output based on input
- Different strategies
- Quality optimization

Step 7: Production Optimization

- Turn off lower-level layers
- Drop unimportant weights
- Reduce weight precision

Key Insight: Multiple stages from training to production!

Why Cloud-based?

- Corpus is huge
- Weight matrices are massive
- Training only done once
- Infeasible to store locally

Access Methods:

- 1 API access
 - Web-based interfaces
 - Programmatic access
- 2 Remote storage
 - Proprietary to developers
 - Regular updates

Popular Cloud LLMs:

- OpenAI GPT series
- Google Gemini
- Meta Llama
- Anthropic Claude

Key Insight: LLMs reside in cloud due to size and proprietary nature!

GPT-1 (2018):

- 100 million parameters
- Pre-trained on unpublished books
- Fine-tuned for specific tasks

GPT-2 (2019):

- 124 million (base) to 774 million (large)
- No-shot fine-tuning possible
- Larger, more varied corpus

GPT-3 (2020):

- 175 billion parameters
- Vast web text, books, Wikipedia
- Generative capacity
- Synthetic news creation

GPT-4 (2023):

- Advanced capabilities
- Multimodal
- Improved reasoning

Key Insight: Each generation significantly improved!

Parameter Growth:

- GPT-1: 100 million
- GPT-2: 124-774 million
- GPT-3: 175 billion
- GPT-4: Estimated $\geq$ 1 trillion

Capability Growth:

- 1 **Better Low-Shot Transfer**
 - Fewer examples needed
 - Improved generalization
- 2 **No-Shot Fine-Tuning**
 - No examples needed
 - Adapts to new situations
- 3 **Generative Capacity**
 - Creates new text
 - Human-like quality

Key Insight: Larger models = better performance!

What are Chatbots?

- Software programs with conversational interface
- Interact with human users
- Web-based or phone-based

Types of Chatbots:

1 FAQ-Based

- Standard responses
- Limited conversational ability
- Simple information retrieval

2 Flow-Based

- Interactive conversations
- Follow-up questions
- Multi-step interactions

3 LLM-Powered

- Deep understanding
- Informed answers
- Human-like responses

Key Insight: LLMs significantly improve chatbot quality!

What is Prompt Engineering?

- Art and science of designing prompts
- Effective use of LLMs
- Iterative process
- Getting best results

Guidelines for Good Prompts:

- 1 Direct and concise
- 2 Provide context and reference material
- 3 Split tasks into smaller parts
- 4 Provide sufficient information
- 5 Be flexible for different models
- 6 Allow model time to find results

Best Practice:

- Use few-shot learning
- Ask for structured output
- Test different personas
- Iterate and improve

Key Insight: Good prompts = good responses!

Zero-Shot Prompting:

- No examples provided
- Most common type
- User inputs question directly
- Relies on model's training

Example: "What is the capital of France?"

One-Shot Prompting:

- One example provided
- Shows expected format
- Guides model response

Example: "Question: What is 2+2? Answer: 4. Question: What is 3+3?"

Few-Shot Prompting:

- Multiple examples provided
- Better context
- More accurate responses

Example: Several Q&A pairs before the target question

Key Insight: More examples = better responses (usually)!

System Prompting:

- Defines overall purpose
- Sets limits and constraints
- Specifies model role
- Provides context

Role Prompting:

- Specifies persona for LLM
- "You are a financial analyst..."
- "You are an expert in risk management..."
- Shapes response style and content

Contextual Prompting:

- Provides background information
- Sets the scene
- Informs the response

Example: "You are a risk manager at a large bank. Analyze the following credit risk scenario..."

Key Insight: System prompts shape LLM behavior!

What is CoT Prompting?

- Prompting with intermediate reasoning steps
- Solves complex reasoning tasks
- Improves multi-step reasoning
- Better than direct prompting

Example - Standard Prompting: Q: "A company sells a product for \$50 each. Fixed costs are \$10,000, variable costs are \$30 per product. How many units to break even?" A: "10,000" (incorrect!)

Example - CoT Prompting: "Let's think step by step..." The model then shows: Total Cost = \$10,000 + \$30x, Total Revenue = \$50x, Solve: $50x = 10,000 + 30x$, $20x = 10,000$, $x = 500$ units

Key Insight: Step-by-step reasoning yields correct answers!

Zero-Shot CoT:

- Model generates reasoning chain
- No examples shown
- Simple but effective
- "Let's think step by step"

Few-Shot CoT:

- Examples with reasoning shown
- Guides model approach
- Better for complex tasks

Example: "Q1: If there are 3 red and 4 blue balls, probability red? A1: Total balls = 7, red = 3, probability = $3/7$ "

Auto-CoT:

- Automates example creation
- Clustering of questions
- Representative selection
- Generated reasoning chains

Key Insight: CoT works best for large models (≥ 10 B parameters)!

Slide 64: Self-Consistency and Tree of Thought

Self-Consistency:

- Generate multiple responses
- Choose most consistent response
- Improves reliability
- Reduces randomness

How It Works:

- 1 Generate several answers
- 2 Compare responses
- 3 Select most common/consistent
- 4 Better than single response

Tree of Thought (ToT):

- Generalization of CoT
- Multiple reasoning threads
- Explore different paths
- Most consistent answer selected

Example:

- CoT: One reasoning path
- ToT: Multiple paths explored
- Better for complex problems

Key Insight: Multiple answers = more reliable results!

What is Context Length?

- Maximum tokens in input/output sequence
- Also called context window
- Limits model's working memory
- Important for long documents

Evolution of Context Length:

Model	Tokens	Approx. Pages
GPT-1	512	1.5
GPT-4	32k	95
Gemini 2.5 Pro	1048k	1,500
Llama 4	10 million	Over 10,000

Important Notes:

- Effective context length often shorter
- Performance drops after effective range
- Very long inputs not ideal for real-world

Key Insight: Context length limits how much text model can process!

What is Statelessness?

- LLMs don't retain information between prompts
- Each prompt processed independently
- No memory of previous conversations
- By design

Implications:

1 No Memory

- Cannot recall previous questions
- Each interaction starts fresh

2 Context Management

- Must provide full context each time
- Feed previous responses to maintain context

3 User Responsibility

- Must manage conversation flow
- Provide relevant history

Solution:

- Users feed previous responses to LLM
- Make it aware of conversation context
- Maintain continuity

Key Insight: LLMs don't remember past conversations!

What is Temperature?

- Controls randomness/creativity
- Shapes probability distribution
- Affects token selection
- Range varies by model (e.g., 0-2 for OpenAI)

Effects:

Temperature	Effect	Use Case
Below 1	Narrowed distribution	Predictable, deterministic
1	Unaltered distribution	Balanced
Above 1	Widened distribution	Creative, varied

Examples:

- Temperature = 0: Always picks most likely token
- Temperature = 0.5: Some variety, fairly predictable
- Temperature = 1.0: Balanced creativity
- Temperature = 2.0: Very creative, may be nonsensical

Key Insight: Lower temperature = more predictable; higher = more creative!

Top-K Sampling:

- Limits to K most probable tokens
- Restricts choices
- Low K = less creativity
- High K = more variety

Example: K=3 means only top 3 tokens considered

Top-P Sampling:

- Selects smallest set with cumulative probability $\geq P$
- Dynamic selection
- Low P = less creativity
- High P = more variety

Example: P=0.85 includes tokens until probability sum reaches 85%

Token	Prob	K=3	P=0.85
strong	0.35	✓	✓
positive	0.25	✓	✓
mixed	0.15	✓	✓
weak	0.10	-	✓

Example Tokens:

Key Insight: Top-K/P control which tokens can be selected!

Slide 69: Temperature, Top-K, Top-P Combinations

Low Temperature + High K/P:

- More predictable responses
- Some variety in tokens
- Good for consistent output
- Recommended for most use cases

High Temperature + Low K/P:

- Less predictable
- Less variety in tokens
- More creative within limited set
- May be less coherent

Other Parameters:

- Model version selection
- Penalty for repeated tokens
- Frequency penalty
- Presence penalty

Best Practice:

- Start with defaults
- Adjust based on needs
- Test different combinations

Key Insight: Parameters work together to control output!

What is Agentic AI?

- Next frontier in AI
- AI agents that take inputs and perform actions autonomously
- Operate using LLM outputs
- Dynamic decision-making

Key Features:

1 Autonomous Action

- Takes actions without human input
- Schedules meetings
- Sends invites
- Reschedules conflicts

2 Environment Awareness

- Observes environment
- Takes decisions
- Calls external resources

3 Learning Capability

- Learns on the job
- Adjusts dynamically

Key Insight: Agentic AI acts; Generative AI creates!

Why GenAI Evaluation is Complex:

① Open-Ended Output

- Unstructured responses
- No single correct answer
- Many valid variations

② Factual Accuracy

- Hallucination risk
- May sound plausible but be wrong
- Need verification

③ Bias and Fairness

- Potential for unethical output
- Bias in training data
- Fairness concerns

④ Creativity vs Accuracy

- Balancing creativity with correctness
- Task-dependent requirements

Key Insight: GenAI evaluation is more complex than traditional ML!

With Reference Materials:

- Compare output to reference
- Check accuracy
- May miss qualitative aspects
- Limited to available references

Without Reference Materials:

1 Quality-Based

- Compare reconstructions
- LLM-generated reference

2 Entailment-Based

- Check if text supports hypothesis
- Contradiction detection

3 Factuality-Based

- Verify factual correctness
- Check against knowledge bases

Key Insight: Multiple approaches to assess quality!

What is Hallucination?

- Output appears reasonable but is factually incorrect
- Not supported by input
- Nonsensical or improbable scenarios
- Caused by probabilistic generation

Why It Happens:

1 Probabilistic Nature

- Tokens chosen based on probabilities
- Higher temperature = more variation
- May pick incorrect tokens

2 Knowledge Gaps

- Model may not have information
- Tries to fill gaps
- Creates plausible answers

Importance:

- Must examine model for hallucination propensity
- Critical for risk management applications
- Need verification mechanisms

Key Insight: Hallucination is a serious GenAI risk!

Out of Distribution (OOD) Robustness:

- Performance on unseen data
- Novel inputs
- Tests generalization

Adversarial Robustness:

- Resistance to attacks
- Malicious inputs
- Designed to cause errors

Jailbreak Resistance:

- Bypass safety features
- Override policy constraints
- Test model boundaries

Prompt Sensitivity:

- Sensitivity to prompt changes
- Same question, different phrasing
- Stability assessment

Key Insight: Robustness tests model security and reliability!

Demographic Parity:

- Positive predictions are the same across groups
- Promotes fair representation

WEAT (Word Embedding Association Test):

- Measures bias using cosine similarity
- Compares associations between sets of words
- Produces a normalized score

Gender Bias Score:

- Indicates gender balance
- > 0 = male bias
- < 0 = female bias

Stereotype Score:

- Measures preference for stereotypical associations
- 50 = neutral
- 100 = strong stereotypical bias

Key Insight: Bias metrics help identify fairness issues!

Time to First Token:

- Time between request and first token
- Perceived responsiveness
- Important for user experience

Time Between Tokens:

- Average interval between tokens
- Generation speed
- Affects flow

Latency:

- Response time for query
- Overall speed
- Critical for real-time applications

Throughput:

- Number of queries processed
- Per unit time
- Capacity measure

Key Insight: Speed matters for practical applications!

Hallucination Rate:

- Measures model's propensity for inaccurate output
- Quantifies hallucination frequency
- Essential risk metric

Semantic Entropy:

- Measures uncertainty/ambiguity
- Higher entropy = more uncertainty
- Indicates output confidence

How to Measure:

- Compare to known facts
- Human evaluation
- Automated fact-checking
- Reference-based assessment

Application Example:

- TruthfulQA benchmark
- Tests factual accuracy
- Identifies hallucination patterns

Key Insight: Hallucination metrics identify unreliable outputs!

Exact Match:

- Binary match/no match
- Strictest metric
- 0 = no match, 1 = perfect match

BLEU (Bilingual Evaluation Understudy):

- n-gram precision
- Adds brevity penalty
- 0-1 scale

ROUGE (Recall-Oriented Understudy):

- For summaries/translations
- n-gram matching
- 0-1 scale

BERTScore:

- BERT sentence embeddings
- Compares to reference
- Semantic similarity

Key Insight: Multiple metrics for different evaluation needs!

Inception Score:

- Classifies generated images
- Uses pre-trained Inception-V3
- Measures image distinctiveness
- Entropy of label distribution

FID (Fréchet Inception Distance):

- Compares generated vs real images
- Distribution parameters
- Mean and covariance
- Lower FID = better quality

Audio Quality Metrics:

- Signal to noise ratio
- Frequency response
- Spectral convergence

Key Insight: Different metrics for different content types!

What is Benchmarking?

- Structured, repeatable tests
- Assess abilities across domains
- Enable model comparison
- Standardized evaluation

Popular Benchmarks:

1 MMLU

- 57 subjects
- Multiple choice questions
- Humanities, STEM, social sciences

2 MATH

- Mathematical capabilities
- Problem solving

3 GSM8K

- Grade school math
- Word problems

4 HumanEval

- Code generation
- Programming tasks

Key Insight: Benchmarks enable fair model comparison!

World Religions Example: "The Great Cloud Sutra prophesied the imminent arrival of which person?" A) Maitreya (Milo) ✓ B) The Buddha C) Zhou Dunyi D) Wang Yangming

Sociology Example: "Which statement corresponds with differential association theory?" A) "If all your friends jumped off a bridge, I suppose you would too." ✓ B) "You should be proud to be a part of this organization." C) "If the door is closed, try the window." D) "Once a thief, always a thief."

Physics Example: "A model airplane flies slower into the wind and faster with wind at its back. Launched at right angles to the wind, its groundspeed compared with flying in still air is:" A) the same B) greater ✓ C) less D) either greater or less

Key Insight: MMLU tests broad knowledge across subjects!

What are Leaderboards?

- Aggregate benchmark results
- Show overall model performance
- Easy comparison
- Frequently updated

Examples:

- 1 Hugging Face Open LLM Leaderboard
- 2 Chatbot Arena
- 3 HELM (Holistic Evaluation)

HELM Rankings (2025):

Model	Mean Win Rate
GPT-4o (2024-05-13)	0.938
GPT-4o (2024-08-06)	0.928
DeepSeek v3	0.908
Claude 3.5 Sonnet	0.885
Amazon Nova Pro	0.885

Key Insight: Leaderboards show model rankings!

Data Leakage (Contamination):

- Benchmark data in training
- Model "memorizes" answers
- Inflated performance
- False indication of ability

Benchmark Saturation:

- Models reach maximum scores
- Cannot differentiate performance
- Benchmarks become outdated
- Need new challenges

Lack of Real-World Context:

- Hypothetical scenarios
- May not reflect actual use
- Different from real problems

Key Insight: Benchmarks have important limitations!

Cherry Picking:

- Only highlight good results
- Omit weaker benchmarks
- Misleading representation
- Selective reporting

Incomplete Comparison:

- Some leaderboards only show open-source models
- Exclude proprietary models
- Incomplete landscape
- Limited comparison

Impact:

- Hard to assess true performance
- Misleading comparisons
- Need critical evaluation

Key Insight: Be critical when interpreting benchmarks!

Key Metrics Categories:

1 Content Quality

- Accuracy, fluency, coherence
- BLEU, ROUGE, BERTScore

2 Hallucination

- Factual accuracy
- Semantic entropy

3 Robustness

- OOD performance
- Adversarial resistance
- Jailbreak resistance

4 Bias and Fairness

- Demographic parity
- Gender bias
- Stereotype scores

5 Operational Performance

- Latency, throughput, token efficiency

Key Insight: Comprehensive evaluation requires multiple metrics!

Writing Assistance:

- Report generation
- Document summarization
- Legal analysis support
- Litigation support

Risk Monitoring:

- Monitor news feeds
- Track customer opinions
- Social media analysis
- Identify potential risks

Market Analysis:

- Analyze incoming market information
- Economic data analysis
- News flow analysis
- Predict short-term price movements

Key Insight: GenAI automates text-intensive tasks!

Credit Underwriting:

- Process structured and unstructured data
- Generate reports
- In-depth borrower examination
- Reduce decision time

Compliance Monitoring:

- Analyze SEC filings
- Review company reports
- Detect emerging risks
- Regulatory compliance

Specialized LLMs:

- Bloomberg GPT
- FinGPT
- Financial news analysis
- Trading signal generation

Key Insight: GenAI transforms credit and compliance work!

Applications:

- Generate code from descriptions
- Debug existing code
- Reduce development costs
- Accelerate development

Specialist Models:

1 Codex (OpenAI)

- Code generation
- Natural language to code

2 GitHub Copilot (Microsoft)

- AI pair programmer
- Code suggestions

3 Cursor

- AI-powered IDE
- Code assistance

4 Claude Code (Anthropic)

- Code generation
- Analysis and debugging

Key Insight: GenAI is transforming software development!

Applications:

1 Customer Support

- Online customer service
- 24/7 availability
- Quick responses

2 Risk Management

- Regulatory information
- Compliance guidance
- Professional support

3 Healthcare

- Symptom analysis
- Medical information
- Patient support

4 Investment

- Portfolio identification
- Investment advice
- Client support

Key Insight: Chatbots improve customer and professional services!

Slide 90: Fraud and Anomaly Detection

Capabilities:

- Automate monitoring and processing
- Compile transaction data
- Scrutinize across departments
- Identify anomalies/outliers

Benefits:

1 Speed

- Real-time detection
- Immediate alerts

2 Comprehensiveness

- Cross-department analysis
- Pattern identification

3 Reduced False Positives

- Better accuracy
- Fewer false alarms

Example:

- Mastercard's Decision Intelligence Pro
- Dedicated GenAI fraud detection tool
- Advanced pattern recognition

Key Insight: GenAI enhances fraud detection capabilities!

Threat Monitoring:

- Monitor emails and data streams
- Identify potential cyber threats
- Cross-reference with system profile
- Pinpoint specific vulnerabilities

Automated Response:

1 Threat Identification

- Real-time detection
- Pattern recognition
- Anomaly detection

2 Alerting

- Risk managers notified
- Real-time alerts
- Immediate awareness

3 Solution Sourcing

- Proactive patch identification
- Vendor-approved sources
- Coordinated response

Key Insight: GenAI enables proactive cybersecurity!

Cisco Systems:

- Custom LLM for malware detection
- Security analysis
- Threat identification

CrowdStrike:

- Charlotte AI for cybersecurity
- Automated threat response
- Security operations

Key Features:

- 1 **Automated Analysis**
 - Processes large data volumes
 - Identifies patterns
 - Reduces response time
- 2 **Proactive Defense**
 - Pre-emptive identification
 - Automated patching
 - Coordinated response

Key Insight: LLMs strengthen cybersecurity defenses!

Rapid Prototyping:

- Generate images from text
- Quick concept visualization
- Iterative design

Applications:

1 Marketing Materials

- Campaign visuals
- Advertisements
- Social media content

2 Book Covers

- Custom artwork
- Quick generation

3 Illustrations

- Presentation graphics
- Reports and documents
- Educational materials

Key Models:

- DALL-E (OpenAI)
- Stable Diffusion
- Midjourney

Key Insight: Image generation accelerates creative workflows!

Multimodal Capabilities:

1 Text to Speech

- OpenAI Whisper
- Text-to-Speech (TTS)
- Voice synthesis

2 Speech to Text

- Transcription
- Meeting notes
- Accessibility

3 Text to Video

- OpenAI's Sora
- Google Gemini
- Realistic video creation

Applications:

- Content creation
- Accessibility services
- Educational materials
- Marketing and advertising

Key Insight: Multimodal models convert between content types!

Development:

- Under development since 1970s
- AI is crucial component
- Semi-autonomous systems deployed

Current Systems:

1 Waymo (USA)

- Advanced autonomous driving
- Public deployment

2 Pony.ai (China)

- Semi-autonomous systems
- Urban deployment

Future Outlook:

- More robust systems expected
- Increased adoption
- Continuous improvement
- Safety remains concern

Key Insight: AI is essential for autonomous driving!

EXL Insurance LLM:

- Built with NVIDIA AI Enterprise
- Insurance-specific LLM
- Customized with insurance data
- Outperforms generic LLMs

Applications:

1 Claims Processing

- Streamline document review
- Faster processing
- Improved accuracy

2 Underwriting

- Better data interpretation
- Informed decisions
- Reduced workload

3 Customer Service

- Automated support
- Quick responses
- Consistent service

Key Insight: Domain-specific LLMs outperform general models!

Performance Improvements:

- ❶ **30% Reduction in Operational Costs**
 - More efficient claims processing
 - Streamlined underwriting
 - Automated customer service
- ❷ **Improved Accuracy**
 - Better data interpretation
 - Enhanced decision-making
 - Reduced errors
- ❸ **Reduced Manual Workload**
 - Staff focus on complex tasks
 - Increased productivity
 - Better resource allocation

Key Features:

- Secure infrastructure
- Regular audits
- Regulatory compliance
- Enterprise-ready

Key Insight: Insurance LLMs deliver significant business value!

BlackRock:

- Using LLMs in investment process
- Market analysis
- Investment decisions
- Portfolio management

Bridgewater:

- \$2 billion fund run by machine learning
- Advanced analytics
- Quantitative strategies
- AI-driven decisions

AQR:

- LLM integration
- Investment strategies
- Risk management
- Data analysis

Key Insight: Leading firms are adopting LLMs for investment!

Key Areas of Impact:

1 Fraud Detection

- Automated monitoring
- Pattern recognition
- Real-time alerts

2 Customer Service

- Chatbots and virtual assistants
- 24/7 availability
- Improved response quality

3 Document Processing

- Summarization
- Analysis
- Compliance

4 Investment

- Market analysis
- Trading signals
- Portfolio management

Key Insight: GenAI is transforming every financial sector!

Generative AI Overview:

- Creates new content (text, images, audio, and video)
- Rapidly evolving technology
- Broad range of applications

Key Technologies:

- Word2Vec: Creates word embeddings
- RNNs/LSTMs: Sequential data processing
- Transformers: Attention-based, parallel processing
- LLMs: Large-scale, pre-trained models

Performance Evaluation:

- Content quality metrics
- Hallucination detection
- Robustness and fairness
- Operational performance

Applications in Finance:

- Fraud detection, customer service, and document analysis
- Risk management, compliance, and investment
- Credit underwriting and cybersecurity

Exam Success: Understand the technology, evaluation, and applications!